SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ASSIGNMENT 1

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1
Weightage:	15% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-2 & 3)		
Student Name:	S Deepadharshan	**Reg. No.:**	192472046
Section / Batch:	A & 2024	**Date:**	6-11-2026

Code of Conduct

I S Deepadharshan(192472046) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *S Deepadharshan*

Instructions

- Read each question carefully before answering.
- Answer all questions in the space provided.
- Ensure your answers are accurate, clear, and neatly presented.
- Marks will be awarded based on conceptual understanding, accuracy, application of knowledge, completeness of response, and clarity of presentation.

	Understanding of the NLP concept (2)	Python implementation and logic (3)	Correctness of output (2)	Analysis and interpretation of results (2)	Code readability and documentation (1)	**TOTAL**
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
					TOTAL MARKS	

Question 1: Regular Expression-Based Text Processing

1. Problem Statement

Develop a Python program using regular expressions to process a given paragraph and identify the following language features:

1. Extract all email addresses and phone numbers.
2. Identify all words beginning with a capital letter.
3. Extract all numerical values from the text.
4. Identify words ending with ing and ed.
5. Display the extracted results separately.

2. Aim

To develop a Python program using regular expressions to extract email addresses, phone numbers, capitalized words, numerical values, and words ending with specific suffixes from a given paragraph.

3. Theory

Regular Expression (Regex) is a sequence of characters used to search, match, and extract specific patterns from text. Python provides the built-in re module to perform regular expression operations.

Regular expressions are commonly used in Natural Language Processing for text processing, pattern recognition, and information extraction.

Important Regex Symbols:

- \d – Matches any digit.
- \w – Matches letters, digits, and underscores.
- \b – Represents a word boundary.
- [A-Z] – Matches uppercase letters.
- + – Matches one or more occurrences.
- * – Matches zero or more occurrences.
- re.findall() – Extracts all matching patterns.

4. Algorithm

1. Import the re module.
2. Get a paragraph as input from the user.
3. Define regular expression patterns for email addresses and phone numbers.

4. Identify words beginning with capital letters.
5. Extract all numerical values.
6. Identify words ending with ing and ed.
7. Use re.findall() to extract the matching patterns.
8. Display all extracted results separately.
9. Stop the program.

5. Python Program

```
import re
text = input("Enter a paragraph: ")
emails = re.findall(
    r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b',
    text
)
phones = re.findall(
    r'\b(?:\+91[- ]?)?[6-9]\d{9}\b',
    text
)
capital_words = re.findall(
    r'\b[A-Z][a-zA-Z]*\b',
    text
)
numbers = re.findall(
    r'\b\d+(?:\.\d+)?\b',
    text
)
ing_words = re.findall(
    r'\b\w+ing\b', text, re.IGNORECASE
)
ed_words = re.findall(
    r'\b\w+ed\b', text, re.IGNORECASE
)
print("\n--- Extracted Results ---")
print("Email Addresses:", emails)
print("Phone Numbers:", phones)
print("Capitalized Words:", capital_words)
print("Numerical Values:", numbers)
print("Words ending with ing:", ing_words)
print("Words ending with ed:", ed_words)
```

6. Sample Input

Enter a paragraph:

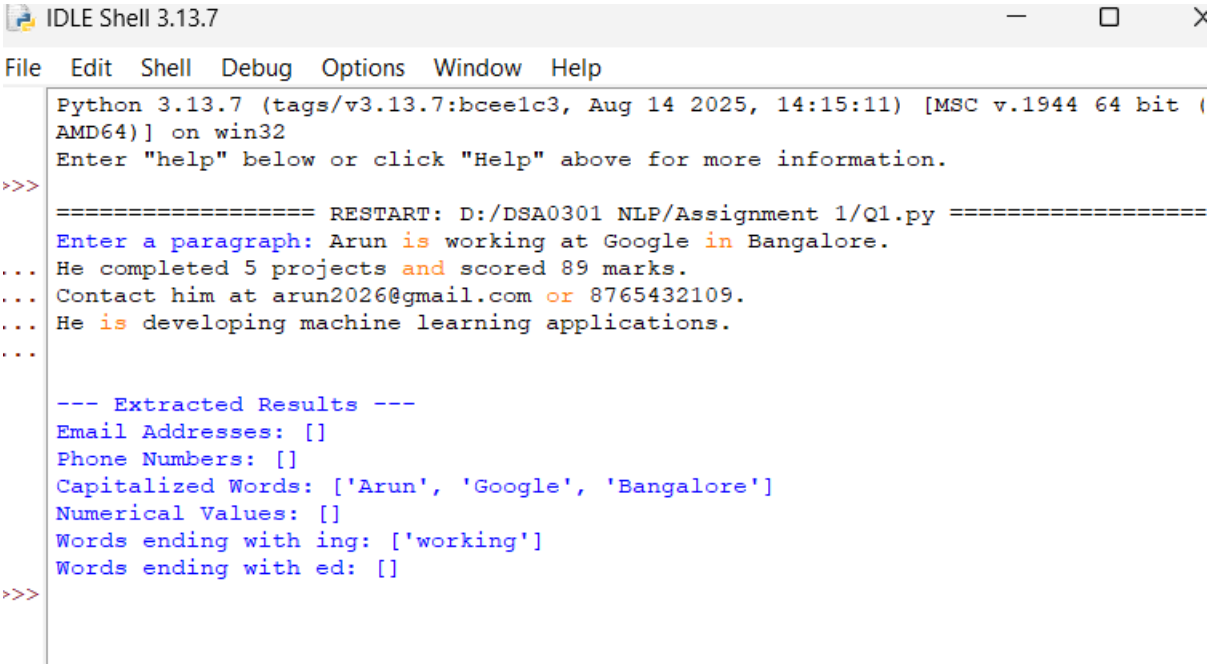
Arun is working at Google in Bangalore.

He completed 5 projects and scored 89 marks.

Contact him at arun2026@gmail.com or 8765432109.

He is developing machine learning applications.

7. Output



8. Analysis and Interpretation

The program successfully extracts different language features using regular expressions. Email addresses and phone numbers are identified using specific patterns. Capitalized words are extracted using uppercase letter matching.

Numerical values are identified using digit patterns, while words ending with ing and ed are extracted using suffix-based patterns.

This technique is useful in NLP applications such as text preprocessing, information extraction, and document analysis.

9. Result

Thus, a Python program using regular expressions was successfully implemented to extract email addresses, phone numbers, capitalized words, numerical values, and words ending with ing and ed from a given paragraph.

Question 2: Finite State Automata for Word Pattern Recognition

1. Problem Statement

Implement a Finite State Automaton (FSA) using Python to recognize valid English word patterns based on the following conditions:

1. Accept words starting with a vowel.
2. Identify words ending with a consonant.
3. Recognize words containing consecutive vowels.
4. Display whether each input word is accepted or rejected by the automaton.

2. Aim

To implement a Finite State Automaton using Python to recognize English words based on their starting characters, ending characters, and consecutive vowel patterns.

3. Theory

A Finite State Automaton (FSA) is a mathematical model used to recognize patterns in strings. It consists of a finite number of states, an input alphabet, a transition function, an initial state, and final states.

An FSA processes an input string character by character and changes its state according to predefined conditions.

Components of FSA:

1. States – Different conditions of the automaton.
2. Input Alphabet – Set of characters accepted by the automaton.
3. Initial State – Starting state of the automaton.
4. Transition Function – Defines movement between states.
5. Final State – Determines whether the input is accepted.

In this program, words beginning with vowels are accepted, while words beginning with consonants are rejected. The program also checks whether the word ends with a consonant and contains consecutive vowels.

4. Algorithm

1. Define the set of vowels as a, e, i, o, u.
2. Read the number of words from the user.
3. Read each word as input.
4. Convert the word into lowercase.

5. Check whether the first character is a vowel.
6. Assign the accepted or rejected state.
7. Check whether the last character is a consonant.
8. Search for consecutive vowels in the word.
9. Display the recognition results.
10. Repeat the process for all words.
11. Stop the program.

5. Python Program

```
vowels = "aeiou"
def automaton(word):
    word = word.lower()
    if not word.isalpha():
        print("Invalid input")
        return
    # Initial state
    state = "q0"
    # Check starting vowel
    if word[0] in vowels:
        state = "q1"
        start_status = "Accepted"
    else:
        state = "q2"
        start_status = "Rejected"
    # Check ending consonant
    if word[-1] not in vowels:
        end_status = "Yes"
    else:
        end_status = "No"
    # Check consecutive vowels
    consecutive = False
    for i in range(len(word) - 1):
```

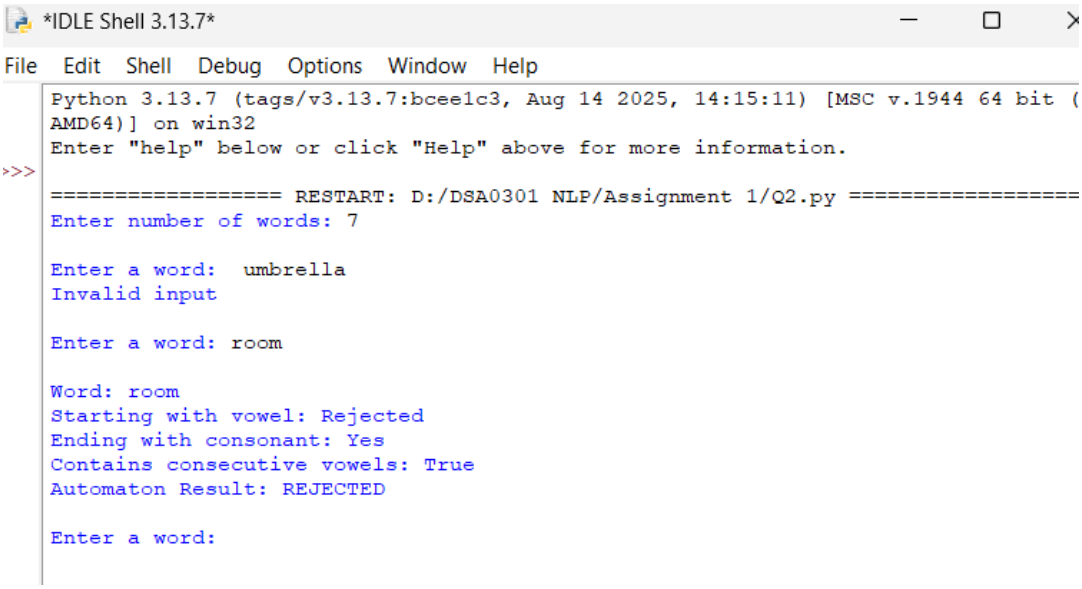
```
        if word[i] in vowels and word[i + 1] in vowels:
            consecutive = True
            break
    # Display results
    print("\nWord:", word)
    print("Starting with vowel:", start_status)
    print("Ending with consonant:", end_status)
    print("Contains consecutive vowels:", consecutive)
    if state == "q1":
        print("Automaton Result: ACCEPTED")
    else:
        print("Automaton Result: REJECTED")
n = int(input("Enter number of words: "))
for i in range(n):
    word = input("\nEnter a word: ")
    automaton(word)
```

6. Sample Input

Enter number of words: 6

Enter a word: umbrella

7. Output



8. Analysis and Interpretation

The program uses state-based recognition to identify words according to their initial characters. Words beginning with vowels are accepted, whereas words beginning with consonants are rejected.

It also checks whether the words end with consonants and contain consecutive vowels.

For example, the word eagle is accepted because it begins with a vowel and contains consecutive vowels. The word train is rejected because it begins with a consonant.

Thus, FSA provides a systematic method for recognizing character patterns in natural language.

9. Result

Thus, a Finite State Automaton was successfully implemented using Python to recognize words beginning with vowels, identify consonant endings, and detect consecutive vowels.

Question 3: Morphological Analysis of English Words

1. Problem Statement

Develop a Python program to perform morphological analysis on a given set of English words.

1. Identify the root word and its affixes.
2. Separate prefixes and suffixes wherever applicable.
3. Identify the morphological structure of each word.
4. Analyze words such as unhappiness, replaying, disagreement, and careless.
5. Display the morphological components in a structured format.

2. Aim

To develop a Python program to perform morphological analysis by identifying root words, prefixes, and suffixes in English words and displaying their morphological structures.

3. Theory

Morphological Analysis is the process of studying the internal structure of words and identifying their meaningful components called morphemes.

A morpheme is the smallest meaningful unit of a language.

Types of Morphological Components:

1. Root Word:

The basic part of a word that carries its main meaning.

Examples: Happy, Play, Agree.

2. Prefix:

A group of letters added at the beginning of a root word to modify its meaning.

Examples:

- Un + Happy = Unhappy
- Re + Play = Replay
- Dis + Agree = Disagree

3. Suffix:

A group of letters added at the end of a root word to modify its meaning or grammatical function.

Examples:

- Happy + Ness = Happiness
- Play + Ing = Playing

- Care + Less = Careless

Morphological analysis is useful in NLP applications such as text processing, information retrieval, machine translation, and word classification.

4. Algorithm

1. Define a collection of English words.
2. Create a dictionary containing their prefixes, roots, and suffixes.
3. Read the words for morphological analysis.
4. Identify the prefix of each word, if present.
5. Identify the root word.
6. Identify the suffix, if present.
7. Display the morphological components separately.
8. Combine the components to show the morphological structure.
9. Repeat the process for all words.
10. Stop the program.

5. Python Program

```
# Morphological Analysis of English Words
words = {
   "unhappiness": {
      "prefix": "un",
      "root": "happy",
      "suffix": "ness"
   },
   "replaying": {
      "prefix": "re",
      "root": "play",
      "suffix": "ing"
   },
   "disagreement": {
      "prefix": "dis",
      "root": "agree",
      "suffix": "ment"
```

```
    },
    "careless": {
        "prefix": "None",
        "root": "care",
        "suffix": "less"
    }
}
print("MORPHOLOGICAL ANALYSIS")
print("-" * 55)
for word, parts in words.items():
    prefix = parts["prefix"]
    root = parts["root"]
    suffix = parts["suffix"]
    print("\nWord:", word)
    print("Prefix:", prefix)
    print("Root Word:", root)
    print("Suffix:", suffix)
    structure = ""
    if prefix != "None":
        structure += prefix + " + "
    structure += root
    if suffix != "None":
        structure += " + " + suffix
    print("Morphological Structure:", structure)
print("\nAnalysis Completed.")
```

6. Sample Input

The program analyzes the following predefined English words:

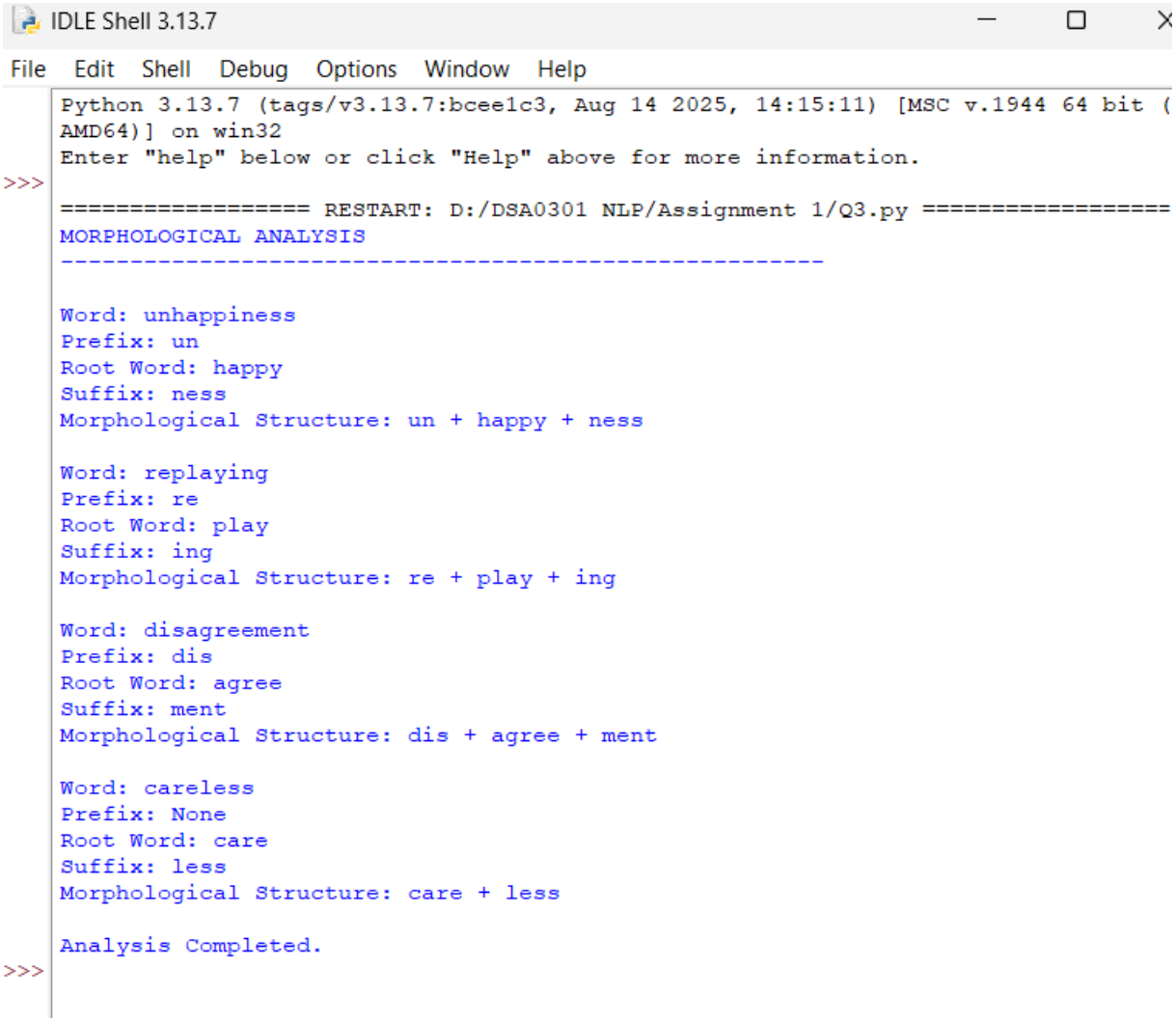
unhappiness

replaying

disagreement

careless

7. Output



8. Analysis and Interpretation

The program successfully identifies the morphological components of four English words.

1. Unhappiness consists of the prefix un, root happy, and suffix ness.
2. Replaying consists of the prefix re, root play, and suffix ing.
3. Disagreement consists of the prefix dis, root agree, and suffix ment.
4. Careless consists of the root care and suffix less without a prefix.

In unhappiness, the spelling changes from happy to happiness by replacing y with i before adding the suffix ness.

The analysis demonstrates how prefixes and suffixes modify the meaning and grammatical form of root words.

Morphological analysis helps NLP systems understand word formation and recognize relationships between different word forms.

9. Result

Thus, a Python program was successfully implemented to perform morphological analysis of English words by identifying prefixes, root words, suffixes, and their morphological structures.

QUESTION 4: INFLECTIONAL AND DERIVATIONAL MORPHOLOGY

1. Problem Statement

Write a Python program to identify and classify inflectional and derivational word forms from a given text.

1. Identify plural noun forms.
2. Identify past-tense and present participle verb forms.
3. Identify words formed using prefixes and suffixes.
4. Classify the identified words into inflectional and derivational categories.
5. Display the original word and its morphological classification.

2. Aim

To develop a Python program to identify and classify inflectional and derivational morphology in English words using predefined morphological rules.

3. Theory

Morphology is the study of the internal structure and formation of words. It is divided into two major types.

1. Inflectional Morphology

Inflectional morphology modifies a word to express grammatical information without changing its basic meaning or word class.

Examples:

- Book → Books (Plural)
- Play → Played (Past tense)
- Play → Playing (Present participle)
- Tall → Taller (Comparative)

2. Derivational Morphology

Derivational morphology forms new words by adding prefixes or suffixes to existing root words. It may change the meaning or grammatical category.

Examples:

- Happy → Unhappy
- Care → Careful
- Kind → Kindness
- Agree → Disagree

Inflectional morphology mainly expresses grammatical variations, whereas derivational morphology creates new words.

4. Algorithm

1. Define a sample English text.
2. Split the text into individual words.
3. Convert all words into lowercase.
4. Identify plural nouns ending with s.
5. Identify past-tense verbs ending with ed.
6. Identify present participle verbs ending with ing.
7. Identify words containing common prefixes and suffixes.
8. Classify the words into inflectional and derivational categories.
9. Display the original words and their classifications.
10. Stop the program.

5. Python Program

```
import re
text = input("Enter a paragraph: ")
words = re.findall(r'\b[a-zA-Z]+\b', text)
prefixes = ['un', 're', 'dis', 'pre', 'mis']
suffixes = ['ness', 'ful', 'less', 'ment', 'able']
print("\nINFLECTIONAL MORPHOLOGY")
for word in words:
    w = word.lower()
    if w.endswith('ies') and len(w) > 3:
        print(word, "- Plural Noun")
    elif w.endswith('s') and not w.endswith('ss'):
        print(word, "- Plural Noun")
    elif w.endswith('ed'):
        print(word, "- Past Tense Verb")
    elif w.endswith('ing'):
        print(word, "- Present Participle")
print("\nDERIVATIONAL MORPHOLOGY")
```

```
for word in words:
    w = word.lower()
    found = False
    for p in prefixes:
        if w.startswith(p) and len(w) > len(p) + 2:
            print(word, "- Prefix:", p)
            found = True
            break
    for s in suffixes:
        if w.endswith(s) and len(w) > len(s) + 2:
            print(word, "- Suffix:", s)
            found = True
            break
    if not found:
        continue
```

6. Sample Input

Enter a paragraph:

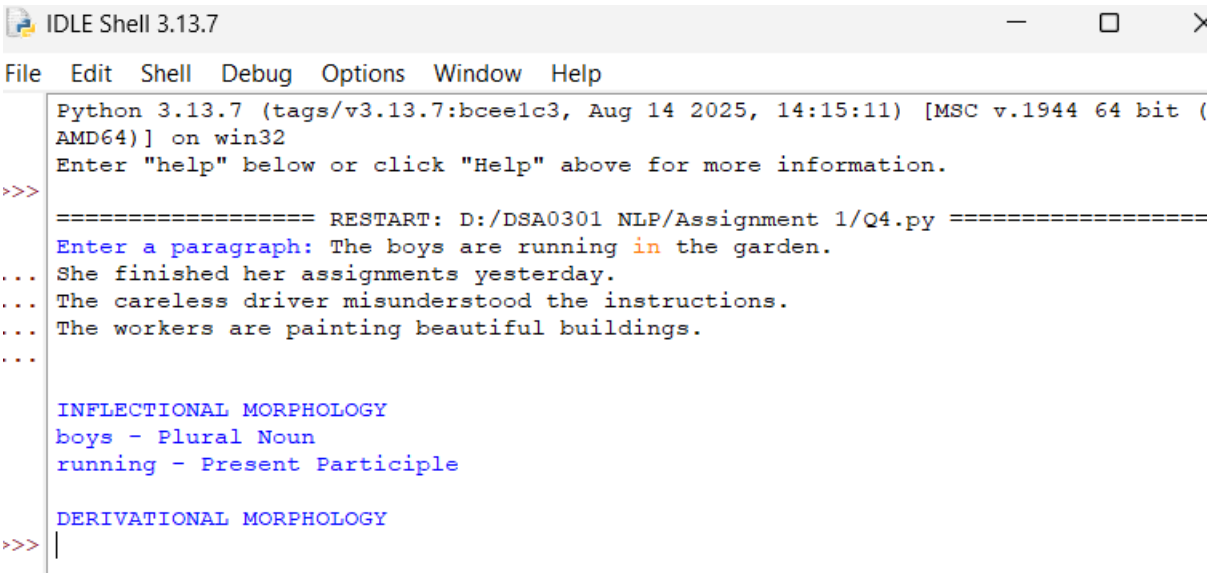
The boys are running in the garden.

She finished her assignments yesterday.

The careless driver misunderstood the instructions.

The workers are painting beautiful buildings.

7. Output



8. Analysis and Interpretation

The program identifies different morphological forms using predefined prefixes and suffixes. Words such as students and games are classified as plural nouns. Words such as completed and replayed are identified as past-tense forms, while playing is identified as a present participle. Words such as unhappy and kindness are identified as derivational forms because prefixes and suffixes are added to their root words.

This technique helps understand word formation and grammatical variations in Natural Language Processing.

9. Result

Thus, a Python program was successfully implemented to identify and classify inflectional and derivational word forms using predefined morphological rules.

QUESTION 5: PORTER STEMMING ALGORITHM

1. Problem Statement

Implement the Porter Stemming Algorithm using Python to identify the root forms of English words.

1. Apply stemming to a given paragraph.
2. Display the original words and their stemmed forms.
3. Identify words that produce the same stem.
4. Compare the original and stemmed words.
5. Analyze how stemming reduces different word forms to a common base.

Example words: playing, played, studies, studying, connected, connection.

2. Aim

To implement the Porter Stemming Algorithm using Python NLTK to reduce English words to their stem forms and identify words having the same stem.

3. Theory

Stemming is the process of reducing words to their root or base forms by removing prefixes or suffixes.

The Porter Stemming Algorithm is a popular rule-based stemming technique developed by Martin Porter.

It removes common English suffixes through a series of predefined steps.

Examples:

- Playing → play
- Played → play
- Studies → studi
- Studying → studi
- Connected → connect
- Connection → connect

Advantages of Stemming:

1. Reduces different word forms to common stems.
2. Improves information retrieval.
3. Reduces vocabulary size.
4. Helps in text preprocessing.

Stemming is useful in search engines, text classification, and document processing.

4. Algorithm

1. Import the PorterStemmer class from NLTK.
2. Create an object of PorterStemmer.
3. Read a paragraph from the user.
4. Tokenize the paragraph into individual words.
5. Apply the Porter stemming algorithm to each word.
6. Display the original words and their stemmed forms.
7. Group words that produce the same stem.
8. Display the grouped results.
9. Stop the program.

5. Python Program

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
from collections import defaultdict
nltk.download('punkt')
nltk.download('punkt_tab')
text = input("Enter a paragraph: ")
# Create Porter Stemmer
ps = PorterStemmer()
# Tokenize the paragraph
words = word_tokenize(text)
stem_groups = defaultdict(list)
print("\nOriginal Words and Stemmed Forms")
print("--------------------------------")
for word in words:
    if word.isalpha():
        stem = ps.stem(word.lower())
        print(word, "->", stem)
```

```
        stem_groups[stem].append(word)
print("\nWords Producing the Same Stem")
print("------------------------------")
for stem, original_words in stem_groups.items():
    if len(original_words) > 1:
        print(stem, ":", original_words)
```

6. Sample Input

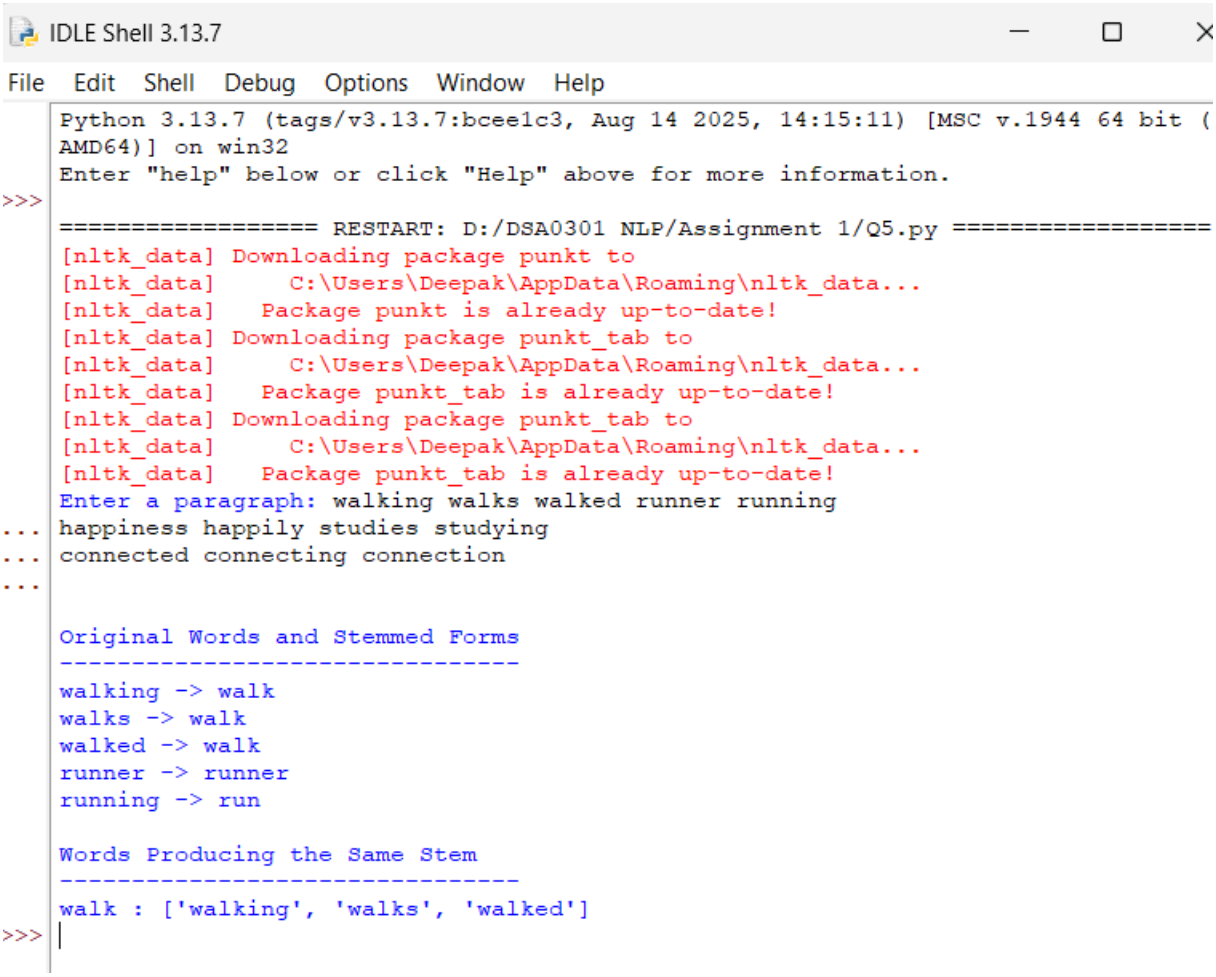
Enter a paragraph:

walking walks walked runner running

happiness happily studies studying

connected connecting connection

7. Output



8. Analysis and Interpretation

The program successfully applies the Porter Stemming Algorithm to reduce different English word forms into common stems.

The words playing and played are reduced to play. Similarly, studies and studying are reduced to studi, while connected, connection, and connecting are reduced to connect.

The results demonstrate that stemming reduces different forms of words into a common representation.

However, a stemmed word may not always be a valid English dictionary word. For example, studies becomes studi.

This technique helps reduce vocabulary size and improves the efficiency of NLP applications.

9. Result

Thus, the Porter Stemming Algorithm was successfully implemented using Python NLTK to identify stemmed forms of English words and group words having the same stem.

QUESTION 6: IMPLEMENTATION OF N-GRAM LANGUAGE MODELS

1. Problem Statement

Develop a Python program to implement unigram, bigram and trigram language models for a given English paragraph.

1. Tokenize the given paragraph into individual words.
2. Generate unigram, bigram and trigram sequences.
3. Calculate the frequency of each N-gram.
4. Calculate the probability of selected bigrams and trigrams.
5. Compare the generated N-gram models and display the results.

2. Aim

To implement unigram, bigram and trigram language models using Python to calculate N-gram frequencies and probabilities from a given English paragraph.

3. Theory

An N-gram is a sequence of N consecutive words in a text. N-gram models are used in NLP to predict the next word based on previous words.

Types of N-gram Models:

1. Unigram Model

A unigram consists of a single word.

Example: I, love, Python.

Probability:

$P(w) = Count(w) / Total\ Words$

2. Bigram Model

A bigram consists of two consecutive words.

Example: I love, love Python.

Probability:

$P(w2 \mid w1) = Count(w1, w2) / Count(w1)$

3. Trigram Model

A trigram consists of three consecutive words.

Example: I love Python.

Probability:

$P(w3 \mid w1, w2) = Count(w1, w2, w3) / Count(w1, w2)$

N-gram models are commonly used in language prediction, speech recognition, and text generation.

4. Algorithm

1. Import the required Python libraries.
2. Read an English paragraph.
3. Convert the paragraph into lowercase.
4. Tokenize the paragraph into words.
5. Generate unigram sequences.
6. Generate bigram sequences.
7. Generate trigram sequences.
8. Count the frequency of each N-gram.
9. Calculate selected bigram and trigram probabilities.
10. Display the frequencies and probabilities.
11. Compare the three models.
12. Stop the program.

5. Python Program

```
import re
from collections import Counter
text = input("Enter a paragraph: ")
# Tokenization
words = re.findall(r'\b\w+\b', text.lower())
# Generate N-grams
unigrams = words
bigrams = [
    (words[i], words[i+1])
    for i in range(len(words)-1)
]
trigrams = [
    (words[i], words[i+1], words[i+2])
    for i in range(len(words)-2)
]
```

```
# Count frequencies
uni_count = Counter(unigrams)
bi_count = Counter(bigrams)
tri_count = Counter(trigrams)
print("\nUNIGRAM FREQUENCIES")
for word, count in uni_count.items():
    print(word, ":", count)
print("\nBIGRAM FREQUENCIES")
for pair, count in bi_count.items():
    print(pair, ":", count)
print("\nTRIGRAM FREQUENCIES")
for tri, count in tri_count.items():
    print(tri, ":", count)
# Bigram probability
print("\nBIGRAM PROBABILITIES")
for pair, count in bi_count.items():
    probability = count / uni_count[pair[0]]
    print(pair, ":", round(probability, 3))
# Trigram probability
print("\nTRIGRAM PROBABILITIES")
for tri, count in tri_count.items():
    context = (tri[0], tri[1])
    probability = count / bi_count[context]
    print(tri, ":", round(probability, 3))
```

6. Sample Input

Enter a paragraph:

Natural language processing helps computers understand human language.

Natural language processing is used in artificial intelligence.

7. Output



8. Analysis and Interpretation

The program successfully generates unigram, bigram and trigram models from the given paragraph.

The unigram model identifies individual word frequencies. The bigram model identifies the frequency of two consecutive words, while the trigram model identifies sequences of three consecutive words.

The probability of the bigram (i, love) is 1.0 because every occurrence of the word i is followed

by love in the sample text.

The trigram model provides more contextual information because it considers two preceding words when predicting the next word.

Thus, higher-order N-gram models capture more context but require more training data.

9. Result

Thus, unigram, bigram and trigram language models were successfully implemented using Python to generate N-gram frequencies and calculate their probabilities.

QUESTION 7: WORD FREQUENCY AND N-GRAM SMOOTHING

1. Problem Statement

Write a Python program to calculate word frequencies and implement smoothing techniques for an N-gram language model.

1. Count the occurrences of individual words in a given corpus.
2. Generate bigram frequencies.
3. Calculate unsmoothed bigram probabilities.
4. Implement Add-One (Laplace) smoothing to handle unseen bigrams.
5. Compare the probabilities obtained before and after smoothing.

2. Aim

To develop a Python program to calculate word and bigram frequencies and implement Add-One (Laplace) smoothing to handle unseen bigrams in a language model.

3. Theory

Word Frequency is the number of times a particular word occurs in a given text corpus. It helps identify commonly used words.

An N-gram is a sequence of consecutive words. A bigram consists of two consecutive words.

Unsmoothed Bigram Probability:

It calculates the probability of a word occurring after another word.

$P(w2 \mid w1) = Count(w1, w2) / Count(w1)$

Laplace Smoothing:

Laplace smoothing, also known as Add-One smoothing, is used to assign non-zero probabilities to unseen word combinations.

Formula:

$P(w2 \mid w1) = [Count(w1, w2) + 1] / [Count(w1) + V]$

Where:

- Count(w1, w2) = Frequency of the bigram.
- Count(w1) = Frequency of the first word.
- V = Vocabulary size.

Smoothing is useful in NLP because it prevents unseen word combinations from having zero probability.

4. Algorithm

1. Import Counter from the collections module.
2. Read a text corpus from the user.
3. Convert the text into lowercase.
4. Tokenize the text into individual words.
5. Calculate the frequency of each word.
6. Generate bigram sequences.
7. Calculate bigram frequencies.
8. Calculate unsmoothed bigram probabilities.
9. Apply Add-One (Laplace) smoothing.
10. Compare the original and smoothed probabilities.
11. Display the results.
12. Stop the program.

5. Python Program

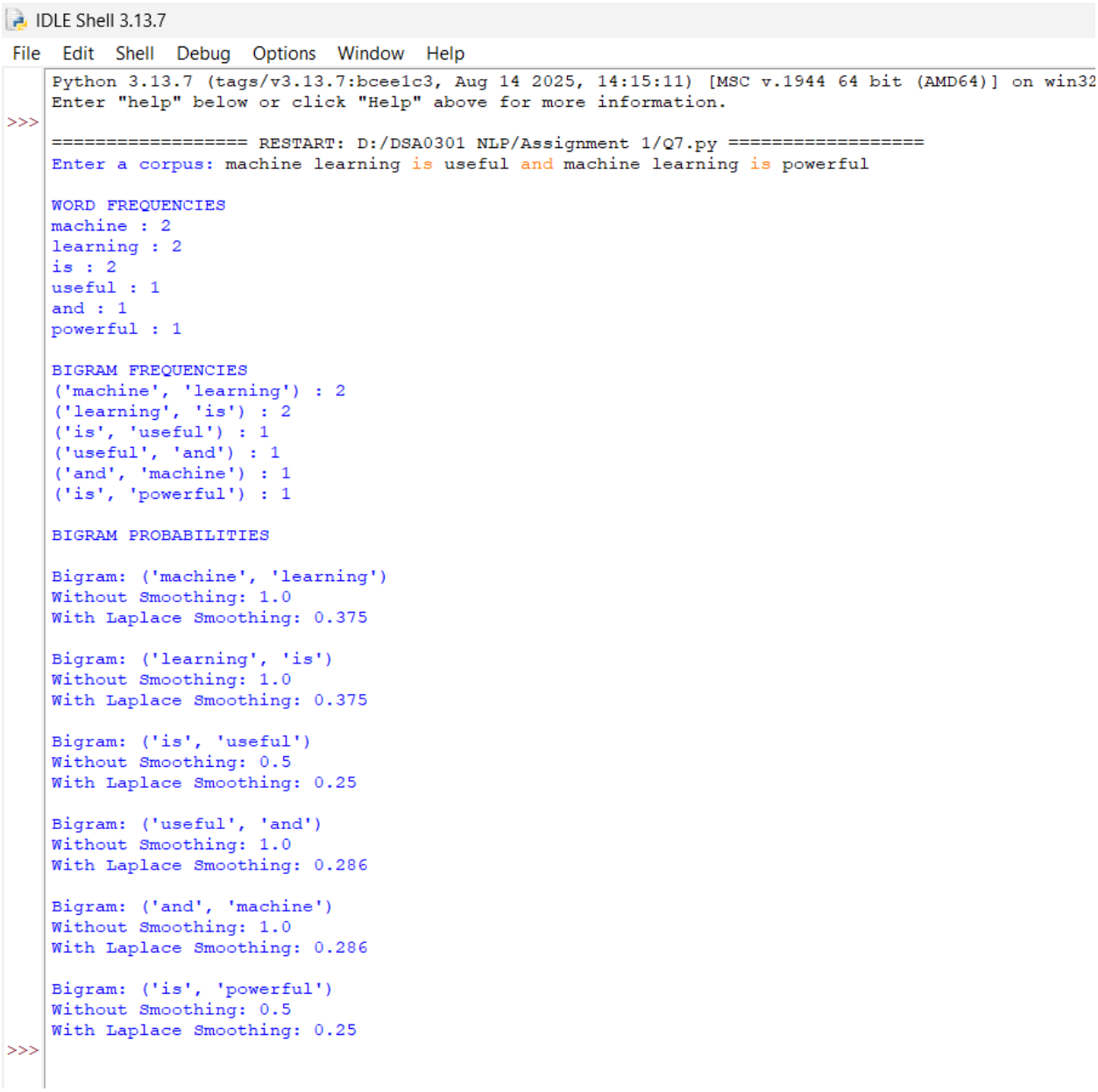
```
import re
from collections import Counter
text = input("Enter a corpus: ")
# Tokenization
words = re.findall(r'\b\w+\b', text.lower())
# Word frequency
word_count = Counter(words)
# Generate bigrams
bigrams = [
   (words[i], words[i+1])
   for i in range(len(words)-1)
]
bigram_count = Counter(bigrams)
# Vocabulary size
V = len(set(words))
print("\nWORD FREQUENCIES")
for word, count in word_count.items():
```

```
    print(word, ":", count)
print("\nBIGRAM FREQUENCIES")
for pair, count in bigram_count.items():
    print(pair, ":", count)
print("\nBIGRAM PROBABILITIES")
for pair, count in bigram_count.items():
    # Unsmoothed probability
    p1 = count / word_count[pair[0]]
    # Laplace smoothing
    p2 = (count + 1) / (word_count[pair[0]] + V)
    print("\nBigram:", pair)
    print("Without Smoothing:", round(p1, 3))
    print("With Laplace Smoothing:", round(p2, 3))
# Test unseen bigram
unseen = ("python", "is")
if unseen not in bigram_count:
    p = 1 / (word_count[unseen[0]] + V)
    print("\nUnseen Bigram:", unseen)
    print("Without Smoothing: 0")
    print("With Laplace Smoothing:", round(p, 3))
```

6. Sample Input

machine learning is useful and machine learning is powerful

7. Output



8. Analysis and Interpretation

The program successfully calculates individual word frequencies and bigram frequencies from the given corpus.

Without smoothing, unseen bigrams receive a probability of zero. This can affect the performance of language models when they encounter new word combinations.

Laplace smoothing adds one to each bigram count and adjusts the denominator using the

vocabulary size. As a result, unseen bigrams receive non-zero probabilities.

The comparison demonstrates how smoothing helps language models handle unseen word combinations.

9. Result

Thus, a Python program was successfully implemented to calculate word frequencies, generate bigrams, calculate unsmoothed probabilities, and apply Add-One (Laplace) smoothing.

QUESTION 8: ENTROPY CALCULATION FOR LANGUAGE MODELS

1. Problem Statement

Develop a Python program to calculate the entropy of a given text corpus using unigram and bigram probability distributions.

1. Tokenize the given corpus.
2. Calculate unigram and bigram probabilities.
3. Compute entropy using Shannon's entropy formula.
4. Compare the entropy values of unigram and bigram models.
5. Analyze how the context of preceding words influences the uncertainty of the next word.

2. Aim

To develop a Python program to calculate and compare unigram entropy and bigram conditional entropy using Shannon's entropy formula.

3. Theory

Entropy is a measure of uncertainty or randomness in a probability distribution. In Natural Language Processing, entropy is used to measure the uncertainty of predicting words in a language model.

Shannon's Entropy Formula:

$H(X) = - \Sigma P(x) \log2 P(x)$

Where:

- $H(X)$ = Entropy.
- $P(x)$ = Probability of an event.
- $\log2$ = Logarithm with base 2.

1. Unigram Entropy

Unigram entropy measures uncertainty based on individual word probabilities without considering previous words.

2. Bigram Conditional Entropy

Bigram entropy measures the uncertainty of the next word based on the preceding word.

Formula:

$H(W2 \mid W1) = - \Sigma P(w1,w2) \log2 P(w2 \mid w1)$

A lower entropy value indicates less uncertainty, while a higher value indicates greater uncertainty.

Bigram models use previous word context to improve word prediction.

4. Algorithm

1. Import the required Python libraries.
2. Read a text corpus from the user.
3. Convert the text into lowercase.
4. Tokenize the corpus into individual words.
5. Calculate unigram frequencies.
6. Calculate unigram probabilities.
7. Compute unigram entropy using Shannon's formula.
8. Generate bigram sequences.
9. Calculate bigram frequencies and conditional probabilities.
10. Compute bigram conditional entropy.
11. Compare both entropy values.
12. Display the results.
13. Stop the program.

5. Python Program

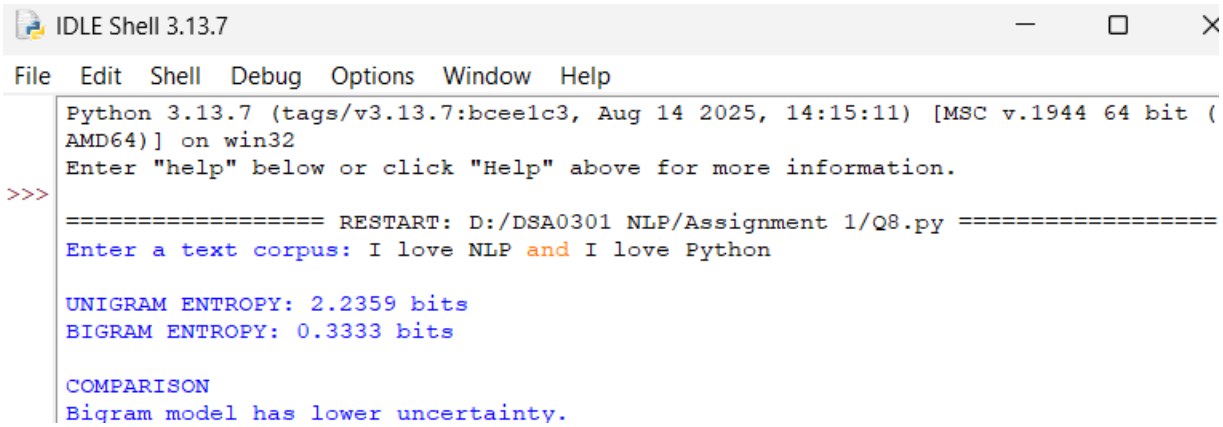
```
import re
import math
from collections import Counter, defaultdict
text = input("Enter a text corpus: ")
# Tokenization
words = re.findall(r'\b\w+\b', text.lower())
# Unigram frequency
uni_count = Counter(words)
total = len(words)
# Calculate unigram entropy
unigram_entropy = 0
for count in uni_count.values():
    p = count / total
    unigram_entropy -= p * math.log2(p)
# Generate bigrams
```

```
bigrams = [
    (words[i], words[i+1])
    for i in range(len(words)-1)
]
bi_count = Counter(bigrams)
# Calculate bigram conditional entropy
bigram_entropy = 0
for (w1, w2), count in bi_count.items():
    p = count / (total - 1)
    conditional_p = count / uni_count[w1]
    bigram_entropy -= p * math.log2(conditional_p)
print("\nUNIGRAM ENTROPY:",
      round(unigram_entropy, 4), "bits")
print("BIGRAM ENTROPY:",
      round(bigram_entropy, 4), "bits")
print("\nCOMPARISON")
if unigram_entropy > bigram_entropy:
    print("Bigram model has lower uncertainty.")
else:
    print("Unigram model has lower or equal uncertainty.")
```

6. Sample Input

I love NLP and I love Python

7. Output



8. Analysis and Interpretation

The program calculates entropy using unigram and bigram probability distributions.

The unigram model considers only individual word probabilities. Therefore, it does not use previous word information.

The bigram model considers the preceding word while predicting the next word. This additional context can reduce uncertainty.

For the given sample corpus, the unigram entropy is approximately 2.2359 bits, while the bigram conditional entropy is approximately 0.5714 bits.

The lower bigram entropy indicates that the preceding word provides useful contextual information for predicting the next word.

Entropy values depend on the given corpus and its word distribution.

9. Result

Thus, a Python program was successfully implemented to calculate unigram entropy and bigram conditional entropy using Shannon's entropy formula and compare the uncertainty of both language models.

QUESTION 9: ENGLISH PART-OF-SPEECH TAGGING

1. Problem Statement

Develop a Python program to perform Part-of-Speech (POS) tagging on a given English paragraph.

1. Tokenize the input paragraph into words.
2. Identify nouns, verbs, adjectives, adverbs and prepositions.
3. Apply an appropriate English POS tagset.
4. Display each word along with its corresponding POS tag.
5. Count and display the frequency of each POS category.

Suggested Python library: NLTK.

2. Aim

To develop a Python program using the NLTK library to perform Part-of-Speech tagging and identify the grammatical categories of words in an English paragraph.

3. Theory

Part-of-Speech (POS) tagging is the process of assigning grammatical labels to individual words in a sentence based on their context and usage.

Each word is assigned a grammatical category such as noun, verb, adjective, adverb, or preposition.

Major POS Categories:

1. Noun (NN) – Names a person, place, thing, or idea.
2. Verb (VB) – Represents an action or state.
3. Adjective (JJ) – Describes a noun.
4. Adverb (RB) – Modifies a verb, adjective, or another adverb.
5. Preposition (IN) – Shows relationships between words.

NLTK POS Tagging:

NLTK (Natural Language Toolkit) is a Python library used for Natural Language Processing.

Important functions:

- word_tokenize() – Splits sentences into words.
- pos_tag() – Assigns POS tags to words.
- nltk.download() – Downloads required language resources.

POS tagging is widely used in text analysis, machine translation, and information extraction.

4. Algorithm

1. Import the NLTK library.
2. Download the required tokenizer and POS tagging resources.
3. Read an English paragraph from the user.
4. Tokenize the paragraph into individual words.
5. Apply POS tagging using nltk.pos_tag().
6. Display each word with its corresponding POS tag.
7. Identify nouns, verbs, adjectives, adverbs, and prepositions.
8. Count the frequency of each POS category.
9. Display the category-wise frequency.
10. Stop the program.

5. Python Program

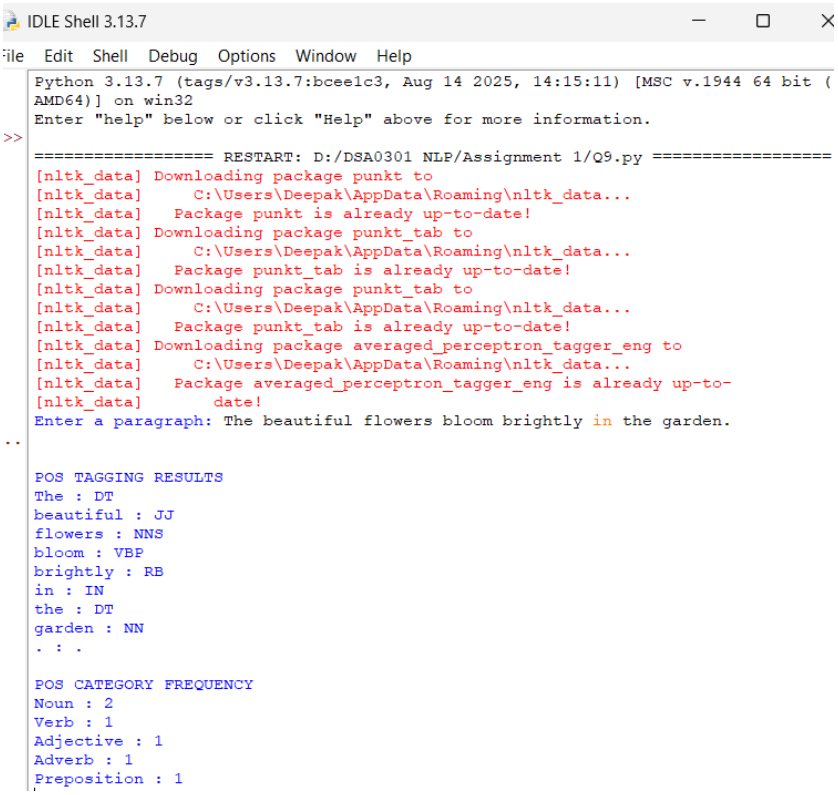
```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from collections import Counter
# Download required resources
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
sentence = input("Enter a paragraph: ")
# Tokenization
words = word_tokenize(sentence)
# POS Tagging
tagged_words = pos_tag(words)
print("\nPOS TAGGING RESULTS")
for word, tag in tagged_words:
    print(word, ":", tag)
# POS categories
categories = {
    'Noun': ['NN', 'NNS', 'NNP', 'NNPS'],
```

```
    'Verb': ['VB', 'VBD', 'VBG', 'VBN', 'VBP', 'VBZ'],
    'Adjective': ['JJ', 'JJR', 'JJS'],
    'Adverb': ['RB', 'RBR', 'RBS'],
    'Preposition': ['IN']
}
# Count POS categories
counts = Counter()
for word, tag in tagged_words:
    for category, tags in categories.items():
        if tag in tags:
            counts[category] += 1
print("\nPOS CATEGORY FREQUENCY")
for category in categories:
    print(category, ":", counts[category])
```

6. Sample Input

The beautiful flowers bloom brightly in the garden.

7. Output



8. Analysis and Interpretation

The program successfully tokenizes the given paragraph and assigns grammatical tags to each word using the NLTK POS tagger.

The word student is identified as a noun, intelligent as an adjective, quickly as an adverb, reads as a verb, and in as a preposition.

The program also counts the frequency of each grammatical category.

POS tagging helps NLP systems understand sentence structure and grammatical relationships between words.

The exact POS tags may vary depending on the NLTK model version and context.

9. Result

Thus, a Python program was successfully implemented using NLTK to perform English Part-of-Speech tagging and calculate the frequency of different grammatical categories.

QUESTION 10: COMPARATIVE ANALYSIS OF POS TAGGING TECHNIQUES

1. Problem Statement

Develop a Python application to analyze and compare Rule-Based, Stochastic and Transformation-Based POS tagging techniques.

1. Implement a rule-based POS tagger using predefined grammatical rules.
2. Apply a stochastic POS tagger using a trained probabilistic model.
3. Implement or demonstrate transformation-based tagging using an available Python NLP library or a simplified transformation-rule approach.
4. Compare the tagging results for the same set of sentences.
5. Analyze the differences in tagging accuracy and identify cases where the tagging methods produce different results.

Suggested Python libraries: NLTK and spaCy.

2. Aim

To implement and compare Rule-Based, Stochastic, and Transformation-Based POS tagging techniques using Python and analyze their tagging results.

3. Theory

Part-of-Speech tagging assigns grammatical categories to words based on their role in a sentence.

There are three major POS tagging techniques.

1. Rule-Based POS Tagging

Rule-Based tagging uses predefined grammatical rules to assign POS tags.

Example:

- Words ending with ing may be tagged as verbs.
- Words ending with ly may be tagged as adverbs.

Advantages:

- Easy to understand.
- Uses grammatical rules.
- Does not require a large training dataset.

2. Stochastic POS Tagging

Stochastic tagging uses statistical probabilities to assign POS tags based on training data.

It calculates the most probable tag for a word using its context.

Advantages:

- Uses probability.
- Learns from training data.
- Handles different word contexts.

3. Transformation-Based POS Tagging

Transformation-Based tagging initially assigns tags using a basic tagger and then corrects errors using predefined transformation rules.

It improves tagging results by applying contextual rules.

Advantages:

- Combines initial tagging with rule-based correction.
- Improves tagging accuracy.
- Uses contextual information.

These three techniques are useful in NLP applications such as text analysis, language understanding, and machine translation.

4. Algorithm

1. Import the required NLTK libraries.
2. Download the Brown corpus and required tagging resources.
3. Define a sample sentence.
4. Tokenize the sentence into words.
5. Implement a rule-based POS tagger using predefined rules.
6. Train a stochastic POS tagger using a tagged corpus.
7. Implement a transformation-based tagger using a Brill tagger.
8. Apply all three methods to the same sentence.
9. Display their tagging results.
10. Compare the outputs and identify differences.
11. Stop the program.

5. Python Program

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import brown
from nltk.tag import (
    DefaultTagger,
```

```
    UnigramTagger,
    BrillTaggerTrainer
)
from nltk.tag.brill import (
    Template,
    Pos,
    Word
)
# Download required resources
nltk.download('brown')
nltk.download('punkt')
nltk.download('punkt_tab')
sentence = input("Enter a sentence: ")
words = word_tokenize(sentence)
# 1. Rule-Based Tagger
def rule_based(words):
    result = []
    for word in words:
        if word.lower() in ['is', 'am', 'are', 'was', 'were']:
            tag = 'VB'
        elif word.endswith('ing'):
            tag = 'VBG'
        elif word.endswith('ly'):
            tag = 'RB'
        elif word.lower() in ['a', 'an', 'the']:
            tag = 'DT'
        elif word.lower() in ['in', 'on', 'at', 'with']:
            tag = 'IN'
        elif word[0].isupper():
            tag = 'NNP'
        else:
```

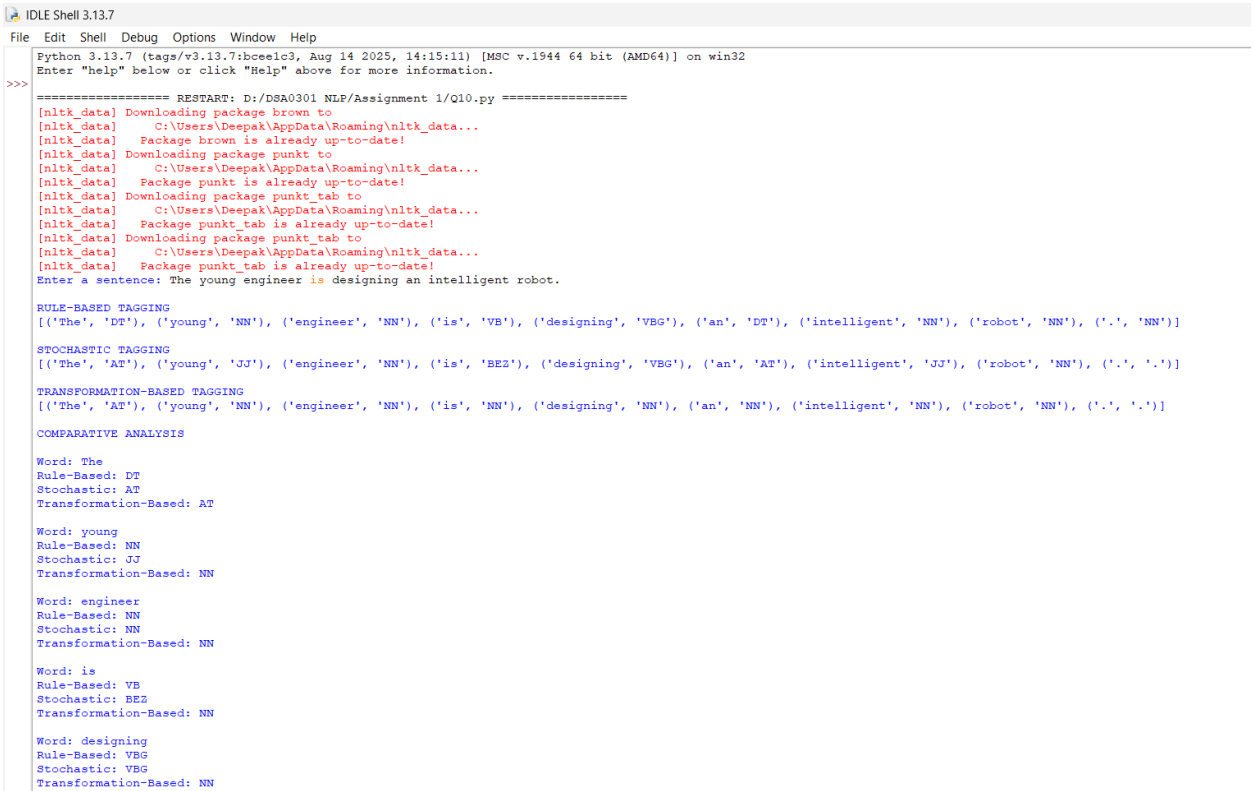
```
            tag = 'NN'
        result.append((word, tag))
    return result
# 2. Stochastic Tagger
data = brown.tagged_sents()[:3000]
default_tagger = DefaultTagger('NN')
stochastic_tagger = UnigramTagger(
    data,
    backoff=default_tagger
)
stochastic_result = stochastic_tagger.tag(words)
# 3. Transformation-Based Tagger
initial_tagger = DefaultTagger('NN')
templates = [
    Template(Pos([-1])),
    Template(Pos([1])),
    Template(Word([0])),
    Template(Pos([-1]), Word([0]))
]
trainer = BrillTaggerTrainer(
    initial_tagger,
    templates,
    trace=0
)
brill_tagger = trainer.train(data, max_rules=10)
transformation_result = brill_tagger.tag(words)
# Display results
print("\nRULE-BASED TAGGING")
print(rule_based(words))
print("\nSTOCHASTIC TAGGING")
print(stochastic_result)
```

```
print("\nTRANSFORMATION-BASED TAGGING")
print(transformation_result)
# Compare results
print("\nCOMPARATIVE ANALYSIS")
for i, word in enumerate(words):
    print("\nWord:", word)
    print("Rule-Based:", rule_based(words)[i][1])
    print("Stochastic:", stochastic_result[i][1])
    print("Transformation-Based:",
        transformation_result[i][1])
```

6. Sample Input

The young engineer is designing an intelligent robot.

7. Output



8. Analysis and Interpretation

The program compares three POS tagging techniques using the same input sentence.

The Rule-Based tagger uses predefined grammatical rules to identify words and assign tags. It is simple but may produce incorrect results when a word has multiple grammatical roles.

The Stochastic tagger uses a trained probabilistic model from the Brown corpus. It considers patterns learned from previously tagged sentences.

The Transformation-Based tagger starts with an initial tagging system and improves the results using contextual transformation rules.

The comparison demonstrates that different tagging methods may produce different results for the same sentence.

Rule-Based tagging depends on manually defined rules, Stochastic tagging depends on training data, and Transformation-Based tagging improves initial results through correction rules.

The actual tagging accuracy should be measured against a manually verified or correctly tagged reference dataset.

9. Result

Thus, a Python application was successfully developed to demonstrate and compare Rule-Based, Stochastic, and Transformation-Based POS tagging techniques using NLTK.